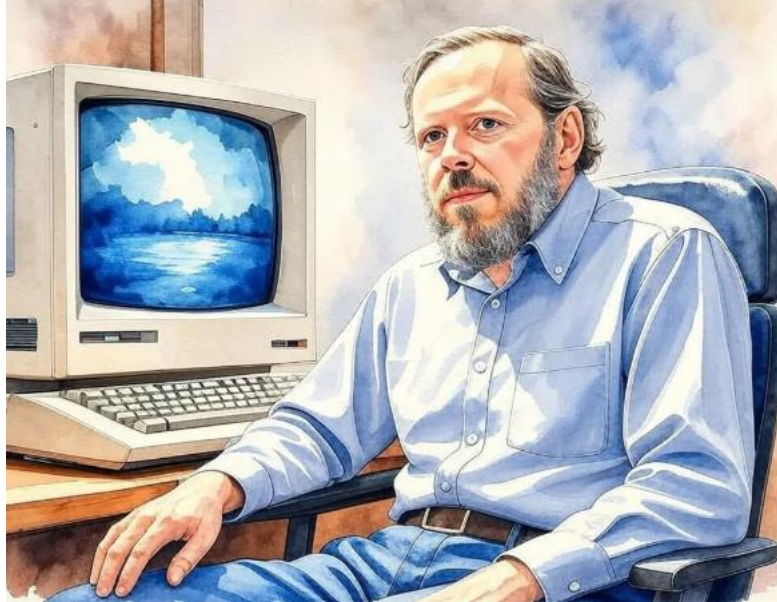


CS25C01- Computer Programming: C

Unit- I INTRODUCTION TO C



The C programming language is a structure oriented programming language developed at AT & T's Bell Laboratories of USA in 1972

It was designed and written by a man named **Dennis Ritchie**

Applications of C programming

- **Operating Systems:** Linux kernel, Microsoft Windows, and Apple macOS/iOS.
- **Databases:** MySQL, PostgreSQL, and Oracle
- **Compilers and Interpreters:** The standard Python interpreter (**CPython**) is written in C. CPython is the actual program that executes your Python code.
- **Embedded Systems:** Microwaves, washing machines, and digital watches.
- **IoT (Internet of Things):** Smart devices, sensors, and home automation systems.
- **Device Drivers:** Graphics cards, keyboards, and printers
- **Game Development:**
- **Graphics and Animation:** Frameworks like **OpenGL** and **DirectX**, which render 2D and 3D graphics for movies and software, are built using C concepts.
- **Web Browsers:** **Google Chrome** and **Mozilla Firefox**

Problem Solving means converting a real-world problem into a step-by-step solution that a computer can understand and execute.

The problem solving process is a systematic way to convert a problem into a working program.

Steps for Problem Solving

Problem Definition (Understand the Problem)

Read the question carefully

Identify:

- **Input**
- **Output**

Clarify what exactly is required

Problem Analysis (Analysing the Problem)

Think about **how to solve it**

Break the problem into smaller parts

Decide:

- What logic is needed?
- Any formulas, conditions, or loops?

Algorithm Design (Pseudo code)

Write step-by-step instructions in simple English

Example:

Start

Read input

Process data

Display result

Stop

Flowchart

Draw diagram using symbols:

- Oval → Start/Stop
- Rectangle → Process
- Diamond → Decision

Helps visualize logic clearly

Coding (Implementation)

- Convert the algorithm into a program (e.g., C language)

Testing & Debugging

- Run program with different inputs
- Check:
 - Correct output?
 - Any errors?

Documentation & Maintenance

- Add comments
- Improve code if needed
- Make the code readable

Real-Life Example: Making Tea

Understanding the Problem

Prepare a cup of tea

- Input → Water, milk, tea powder, sugar
- Output → Ready tea

Analysing the Problem

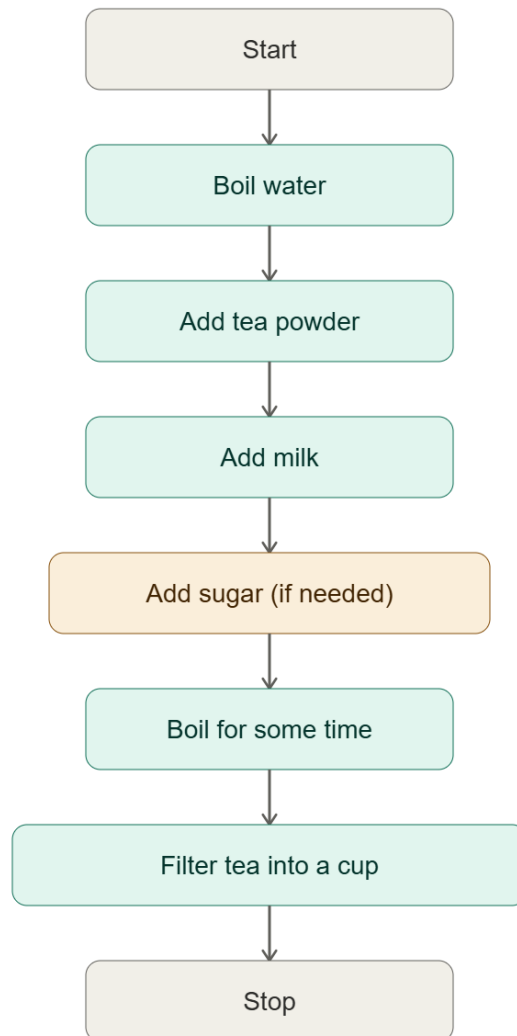
Think:

- What steps are needed?
- In what order?
- Any conditions? (e.g., sugar optional)

Designing the Solution (Algorithm)

1. Start
2. Boil water
3. Add tea powder
4. Add milk
5. Add sugar (if needed)
6. Boil for some time
7. Filter tea into a cup
8. Stop

Flowchart



Coding (Implementation)

Prepare the tea as per the flowchart

Testing & Debugging

- Taste the tea
- Too strong → reduce tea powder
- Less sweet → add sugar

This is like fixing errors in a program!

Maintenance

- Next time:
 - Improve taste
 - Adjust ingredients

Problem: Find the Square of a Number

Understanding the Problem

- What is required? → Find the **square of a number**
- Input → A number n
- Output → $n \times n$ (square)

Example:

If $n = 5$, output = 25

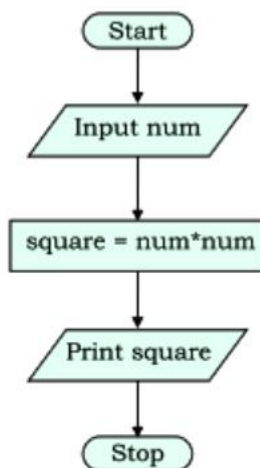
Analyzing the Problem

- We need:
 - One variable $\rightarrow n$
 - One operation $\rightarrow$ multiplication

Designing the Algorithm

1. Start
2. Read the number n
3. Compute square = $n \times n$
4. Display square
5. Stop

Flowchart



Coding (Implementation)

```
#include <stdio.h>
int main() {
    int n, square;
    printf("Enter a number: ");
    scanf("%d", &n);
    square = n * n;
    printf("Square = %d", square);
    return 0;
}
```

Testing & Debugging

Test Case 1:

Input: 4

Output: 16

Test Case 2:

Input: -3

Output: 9

Maintenance

- Improve program:
 - Handle large numbers
 - Add comments
 - Use meaningful variable names



Find the Area of a Circle

Problem: Calculate the area of a circle.

- **Input:** Radius r
- **Process:** $\text{area} = 3.14 \times r \times r$
- **Output:** Area of the circle

Example:

Input: 5

Output: 78.5